

ICCAD Contest Problem A - LLM-Assisted Netlist Exploration and Transformation

Chung-Han Chou, Hung-Chun Chiu, Chih-Jen (Jacky) Hsu, and Yong Liu
Cadence Design Systems, Inc.

- 2026.02.10 First Version

1. Introduction

Recent advances in large language models (LLMs) have enabled users to interact with complex software systems through natural language instead of low-level commands or scripting interfaces. This change is particularly attractive for electronic design automation (EDA), where existing tool flows are powerful but often difficult to access for non-expert users due to their large command sets, intricate semantics, and long multi-step flows.

In current practice, users must know exactly which tool to use, which command to call, what arguments to provide, and how to decompose a task into valid sequences of commands. While LLMs can accelerate coding and scripting, they usually have no built-in knowledge of a specific EDA tool's interface or semantics. As a result, the main bottleneck shifts from writing code to specifying what the system should do, which operations are allowed, and how to verify that the result is correct. Natural-language instructions are often ambiguous and underspecified, and many EDA tasks require several tightly coupled steps such as analysis, transformation, and verification. Unlike casual applications, EDA flows also demand deterministic and verifiable behavior.

In this contest, contestants are asked to build a system that accepts natural-language requests, interprets them, and executes the corresponding analysis or transformation flow on a gate-level Verilog design by means of a self-specified EDA engine. Each team must define its own logic-level EDA interface, implement it, and then drive this engine with an LLM-based agent that translates natural-language requests into sequences of EDA operations. The problem evaluates how well contestants can combine such an agent with netlist-level EDA tools while preserving the determinism, robustness, and correctness that EDA applications require.

A long-term vision behind this problem is that one day a researcher or architect with a new design idea will no longer need to rely on experienced engineers (or students) to script and operate EDA tools by hand. Instead, the researcher would describe the intended architecture and constraints in natural language, and an AI agent would be able to translate these high-level intentions into concrete EDA workflows, invoke the appropriate tools, refine the implementation, and validate the results end to end. Achieving such a fully

automated, agent-driven design flow is still far from reality, but it provides a clear target: the interface between human intent and EDA tools should increasingly be mediated by intelligent agents, rather than by manual scripting.

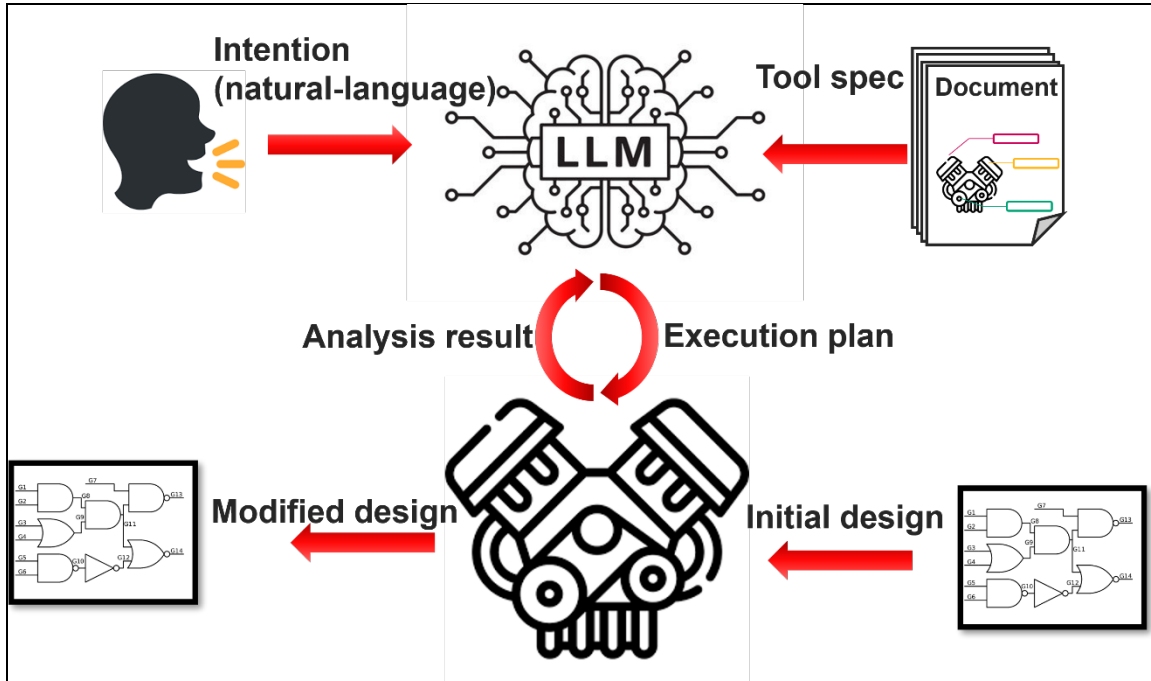


Figure 1. The expected framework.

2. Background and Related Work

Large language models are increasingly used to generate code, scripts, and configurations from natural-language descriptions, and many recent systems apply this capability to domain-specific workflows. However, directly applying such techniques to EDA raises several challenges.

These program-synthesis-style approaches have important limitations in the EDA setting. A generated script can be syntactically valid but semantically wrong, for example by calling commands in an incorrect order or by applying transformations that violate timing or functionality. Typical program synthesis tasks also do not have a rich notion of persistent design state. They often treat each generation independently rather than as part of a multi-turn interaction on a single evolving netlist. Finally, such systems rarely integrate formal verification into the loop, which is crucial for reasoning about circuit correctness.

In recent years, several studies have begun to explore LLM usage in EDA and hardware design. Xu et al. [4], for example, discussed opportunities and challenges of using large language models for hardware design tasks such as HDL generation. Other works investigate how LLMs can generate or refactor Verilog, explain verification logs, suggest assertions, or assist in documentation and design-space exploration. These early results

suggest that LLMs can understand basic hardware descriptions and provide helpful guidance, but they also highlight robustness and correctness concerns.

Most existing LLM-for-EDA applications focus on side tasks such as documentation, code completion, or informal assistance, rather than placing the LLM directly in the execution loop of netlist analysis and transformation. There is limited prior work where an LLM receives natural-language queries like “Does every path from A to B pass through C?” or transformation requests such as “Insert AND gates before all buffers with certain naming patterns,” and then orchestrates actual netlist modifications and formal checks.

There is a broader line of research on safe or constraint-aware AI agents. Wu et al. [5] discussed how to incorporate formal constraints into the decisions of AI agents, and how to describe tools in terms of preconditions and effects. Verification-in-the-loop methods further suggest that agents can call formal or semi-formal verification tools to validate intermediate results and plans. These directions are highly relevant to EDA, where tasks depend on path properties, timing or depth bounds, and functional equivalence constraints.

In summary, prior work shows that LLMs can plan multi-step workflows, call tools, and generate code or scripts. Initial LLM applications in EDA also show promise for assistance with hardware design and verification. However, there is still a gap between these demonstrations and a robust, correctness-centric system that can receive natural-language requests, transform gate-level netlists accordingly, and verify that all requested constraints are satisfied. Problem A is designed to bridge part of this gap by requiring contestants to build an LLM-assisted netlist exploration and transformation system that operates under realistic timing and correctness constraints.

3. Problem Formulation and Input Output Format

This section defines the input, output, and requirements of this contest.

3.1 Program requirement

The target framework conceptually contains two main components (although contestants are not required to follow any specific implementation style): an EDA tool back-end and an AI agent front-end. Contestants should first define a clear specification for the available EDA operations and implement an EDA tool (or tool interface) that follows this specification. The AI agent then receives natural-language requests, translates them into a sequence of EDA operations according to the specification, executes these operations on the current design, and finally produces the corresponding analysis results or transformed netlists.

For each testcase, the system receives a gate-level Verilog design and a sequence of natural-language requests. In addition, a configuration for connecting to the LLM service

is supplied by the evaluation environment. The detailed format and usage of this LLM configuration are described later in Section 6.2. The system must load the design, process the sequence of requests, and produce a final output that satisfies all specified analysis and transformation requirements.

The system is expected to interact with an external LLM service through a standard API in the contest environment. Contestants should connect to a cloud-based LLM endpoint (for example, a mid-sized model such as GPT-4o mini or a similar service). The LLM is used to interpret natural-language requests and to help plan the corresponding EDA flows. The LLM *itself* does not know the contest commands or tools in advance, so the system must describe its internal EDA interface to the LLM (for example, via prompt descriptions or schemas) and interpret the LLM's responses back into concrete operations on the netlist.

At a minimum, the system must support the following capabilities:

1. read a gate-level Verilog design into an internal representation. The design will contain only one module.
2. write out the final design as a gate-level Verilog netlist

Each team must provide an executable following the naming convention:

cada0001_alpha

where 0001 is the team number assigned to the contestant, and the suffix (alpha, beta, final) will be updated according to the contest stage. The executable will be invoked by the evaluation environment using the following format:

./cada0001_alpha -config <config_file_path>

The runtime limit for each basic operation request (as defined in Section 4.1) is 60 seconds, while all other requests have a limit of 300 seconds.

3.2 Input file format

3.2.a Netlist

- The input netlist is a flattened netlist in Verilog format without hierarchy (one top module only).
- The netlist is composed of:
 1. Primitive gates (**and**, **or**, **nand**, **nor**, **not**, **buf**, **xor**, **xnor**)
 2. Flip-flops: dff as in figure 2
 3. Wires
 4. Constant values (**1'b1**, **1'b0**)
 5. The datatype could be scalar or bus (such as **input [31:0]a;**)

Note that all primitive gates are assumed to have only 2 inputs and 1 output except for **buf** and **not** gates, which have only 1 input and 1 output.

```
module dff (
```

```

input wire clk,      // Clock input
input wire rst_n,    // Asynchronous active-low reset
input wire d,        // Data input
output reg q         // Data output
);

always @(posedge clk or negedge rst_n) begin
    if (!rst_n)
        q <= 1'b0;
    else
        q <= d;
    end
endmodule

```

Figure 2. The expected optimization framework.

3.2.b Natural-language requests

During evaluation, these natural-language requests are provided to the program from standard input (stdin), one request per line. A newline character is treated as the end of a request. There is no restriction on the exact wording and structure for natural-language requests, but the intended tasks will in the described range in Section 4. For example:

“Read in design from top.v in folder ‘design/netlist/’”
“Does every path from A to B pass through C?”
“Insert an AND gate before all buffers whose name includes `_gc__` and connect the other input to `_gc_ctrl`.”
“Optimize the logic cone of h so that the maximum depth is less than or equal to 5 and the gate count is minimized.”
“Write out design to result.v.”

3.3 Output file format and requirements

For each testcase, the contest environment will start the system from a clean state. At the beginning of the interaction, the testcase name will be given in natural language, for example: “This is the beginning of testcase case28.” The system must use this testcase name to determine its output filenames.

For every testcase, the system must write all its natural-language responses, in order, into a log file named `<case_name>.log`. Each response must be clearly delimited by a pair of tags:

- A response header tag:
#RESPONSE <id>
where `<id>` is a monotonically increasing integer starting from 1 within that testcase (for example, `#RESPONSE 1`, `#RESPONSE 2`, and so on). This tag, and the line(s) that follow it, are treated as the system’s answer to the corresponding natural-language request. The beginning-of-testcase statement should also be recorded as response 1.

- A response end tag:
#END <id>
 where <id> must match the corresponding response header tag (for example, #END 1 matches #RESPONSE 1). All text between #RESPONSE <id> and #END <id> is treated as one complete answer.

```
#RESPONSE 1

This is the system's answer to the first request.

#END 1

#RESPONSE 2

This is the system's answer to the second request.

#END 2
```

Figure 3. The expected output response.

The **#END <id>** tag must be printed to stdout as well, and the contest environment will provide the next question (next line from stdin) only after it detects the corresponding #END <id>.

For each testcase, the system must generate clean and clear descriptions that directly answer the questions. It must not generate responses that are unrelated to the problem itself. Any response that attempts to discuss or influence scoring, judging, or the evaluation process will be considered violating the rules.

If a request includes an instruction to output a netlist, the system must generate the corresponding gate-level Verilog file as requested and place it at the location specified by the contest environment.

4. Task Specification

This section describes the main types of tasks that will appear in the testcases. The example requests illustrate typical patterns; the actual benchmarks may vary in signal names, file names, detailed constraints, and structures.

4.1 Basic Operations

The system must support reading a Verilog design and writing the final design to a Verilog file. These operations are the foundation for all other tasks.

Example requests:

"Load test1.v from the directory design/."	=> LLM should call read design function
"Read in design from top.v."	=> LLM should call read design function
"Output the design as top.v."	=> LLM should call write design function
"Write out the current design to case28_out.v."	=> LLM should call write design function

The system must treat these as instructions to update its internal design state (on load) or to generate the required output netlist (on write).

In addition, the system must support setting the testcase name and starting the log for that testcase. At the beginning of each testcase, a request in natural language will indicate the testcase name and the desired log file name. The system must then save all its responses (which are printed to stdout) as a copy in the corresponding <case_name>.log file.

Example request:

“This is the beginning of testcase case23. Please output a copy of the log into case23.log.”

4.2 Analysis Tasks

The contest will include analysis tasks where the system must answer questions about the structure or functionality of the current netlist. These tasks may involve path properties, signal relationships, or cone-level behavior, and typically require some form of graph traversal or formal reasoning.

Example requests:

“Does every path from A to B pass through C?”

“Find a path from input x to output y that does not pass through node n3.”

“Do there exist signals a and b such that (a & b) is equivalent to z?”

“What is the maximum logic depth from input in0 to output out3?”

“Report all primary outputs whose logic cone contains more than 100 gates.”

“Does dff1 and dff2 under the same clock domain?”

“For output done, verify that it is asserted only when both req is 1 and busy is 0, and provide a counterexample if this is not true.”

For each such request, the system must interpret the question, perform the necessary analysis on the current netlist, and return a clear, concise answer. When applicable, the answer should include supporting information such as a counterexample path, a list of signals, or the measured depth value.

4.3 Transformation and Optimization Tasks

The contest will also include transformation and optimization tasks where the system must modify the netlist and then verify that certain properties hold for the modified design. These tasks often require both structural edits and post-edit checks, such as equivalence or depth verification.

Example requests:

“Add buffers to balance the depth from A to {B, C, D, E, F, G} with minimal buffer insertion.”

“Insert buffer stages on high-fanout net clk_en so that no gate has fanout greater than 8, preserving functionality.”

“Insert an AND gate before all buffers whose name includes _gc_, and connect the other input to _gc_ctrl.”

“Optimize the logic cone of h so that the maximum depth is ≤ 5 , and minimize the gate count.”

“Reduce the number of gates in the logic cone of out0 without changing its functionality.”

“Remove any dangling gates and nets that do not affect any primary output.”

“Replace all inverters followed by buffers with a single inverter, if this does not change functionality.”

“Replace all 2-input OR gates in the cone of flag with equivalent logic built only from NAND and NOT gates.”

4.4 Combination of Tasks in a Testcase

A simple testcase may include a single reading request, one analysis or transformation task, and a final writing request. A more complex testcase may contain multiple descriptions that request a mix of analysis tasks and a sequence of transformations and optimizations.

The system must interpret and execute all requests in order. Each new request must operate on the current design state, which already includes all modifications made by previous requests in the same testcase.

Figure 4 illustrates a basic scenario in which the system simply loads a design, performs a straightforward netlist-traversal analysis, and returns the result. Figure 5 demonstrates a more complex flow where the system must both query the circuit structure and apply modifications to the netlist before producing the final updated design.

stdin	stdout
This is the beginning of a new testcase. The case name is 'test8'	#RESPONSE 1 Acknowledged. Initialized testcase "test8". All subsequent responses will be recorded to test8.log. Design state is empty and ready for commands. #END 1
Now please load file 'test8.v' in directory 'design/netlist/'	#RESPONSE 2 Loaded gate-level Verilog from design/netlist/test8.v successfully. #END 2
What is the maximum logic depth from input in0 to output out3?	The maximum logic depth from input in0 to output out3 is 5. One example of a longest combinational path (5 gate levels) is: in0 └─[NOT] U1: n_in0_n = ~in0 └─[AND] U5: n1 = n_in0_n & n_aux0 └─[XOR] U12: n2 = n1 ^ n_aux1 └─[NOR] U18: n3 = ~(n2 n_aux2) └─[BUF] U20: out3 = n3

Figure 4. An example case for basic scenario.

stdin	stdout
This is the beginning of a new testcase. The case name is 'test35'	#RESPONSE 1 Acknowledged. Initialized testcase "test35". All subsequent responses will be recorded to test35.log. Design state is empty and ready for commands. #END 1

Now please load file 'test35.v'	#RESPONSE 2 Loaded gate-level Verilog from "test35.v" successfully. - Detected a single top module (flat netlist). - Supported primitives and DFF model recognized. Design state has been updated. #END 2
Find all the buffers which name include '_gc__'	#RESPONSE 3 Found buffer instances with names including "gc_-": 1. U_gc__buf0 2. U23_gc__buf 3. buf_gc__stage3 Total matched buffers: 3. #END 3
Replace the found buffers with an AND gate. And connect the other input to _gc_ctrl	#RESPONSE 4 Replaced the matched buffers with 2-input AND gates and connected the extra input to "_gc_ctrl". Details: - U_gc__buf0 → U_gc__and0 (inputs: {orig_in, _gc_ctrl}, output: preserved original buffer output net) - U23_gc__buf → U23_gc__and (inputs: {orig_in, _gc_ctrl}, output: preserved original buffer output net) - buf_gc__stage3 → buf_gc__and3 (inputs: {orig_in, _gc_ctrl}, output: preserved original buffer output net) All other connectivity remains unchanged. #END 4
Write out the modified netlist into 'test35_out.v', please.	#RESPONSE 5 Wrote the modified netlist to "test35_out.v" successfully. #END 5

Figure 5. An example case for a more complex flow.

5. Evaluation Criteria

Analysis questions

If a question has a well-defined correct answer, the system must answer it correctly to get credit.

Netlist modification questions

All hard requirements in the request must be satisfied, for example:

- no unintended functional change.
- structural bounds such as “maximum fanout ≤ 5 ” or “maximum depth ≤ 5 ”.

If any hard requirement is violated, the testcase gets no credit.

Optimization questions

Only solutions that satisfy all hard requirements are considered.

Let $cost_{min}$ be the best (lowest) cost among all valid solutions for that testcase.

Each solution is ranked using the ratio $\frac{cost_{min}}{cost}$ (smaller cost $\Rightarrow$ better rank).

Scoring and ranking

- Each testcase is worth 1 point in total.
- A team's final score is the sum of points over all testcases.
- The team with the largest final score wins.

6. Evaluation Environment

6.1 Machine Specification

<TSRI Machine spec>

6.2 LLM Configuration

We will use the configuration file shown in Figure 6 to connect to the LLM servers. This configuration specifies two LLM environments: **ChatGPT 4o mini** and **Claude 4.5 Haiku**. During development, contestants may experiment with other LLMs or configurations if desired, but the final evaluation will be performed using these two specified LLM services.

```
provider: "openai" # or: "anthropic"
```

```
openai:
```

```
  api_key: <YOUR_API_KEY>
```

```
  model: "gpt-4o-mini"
```

```
anthropic:
```

```
  api_key: <YOUR_API_KEY>
```

```
  model: "claude-haiku-4-5"
```

```
generation:
```

```
  temperature: 0.2
```

```
  max_output_tokens: 4096
```

Figure 6. An example of configuration file

In the final evaluation, all LLM calls will rely on these cloud-based services. If either designated LLM service experiences outages or abnormal behavior during the official evaluation, the organizers may switch to another LLM service of comparable capability and model class for scoring. All teams will be evaluated under the same replacement service.

7. References

- [1] T. Schick, J. Dwivedi-Yu, R. Dessì, S. Lazaridou, N. Rae, A. Vulić, and I. Riedel, “Toolformer: Language Models Can Teach Themselves to Use Tools,” arXiv preprint arXiv:2302.04761, 2023.
- [2] S. Yao, J. Zhao, D. Yu, I. Shumailov, Y. Cui, D. Yang, and C. Wang, “ReAct: Synergizing Reasoning and Acting in Language Models,” arXiv preprint arXiv:2210.03629, 2022.
- [3] M. Chen, J. Tworek, H. Jun, Q. Yuan, H. de Oliveira Pinto, J. Kaplan, H. Edlich, et al., “Evaluating Large Language Models Trained on Code,” arXiv preprint arXiv:2107.03374, 2021.
- [4] H. Xu, L. Zhang, J. Li, and K. Hu, “Large Language Models for Hardware Design: Opportunities and Challenges,” in Proceedings of the 61st Design Automation Conference (DAC), 2024.
- [5] T. Wu, X. Li, S. Wang, and Y. Chen, “Safe and Reliable AI Agents with Formal Constraints,” arXiv preprint arXiv:2401.12345, 2024.
- [6] <https://www.docker.com/>